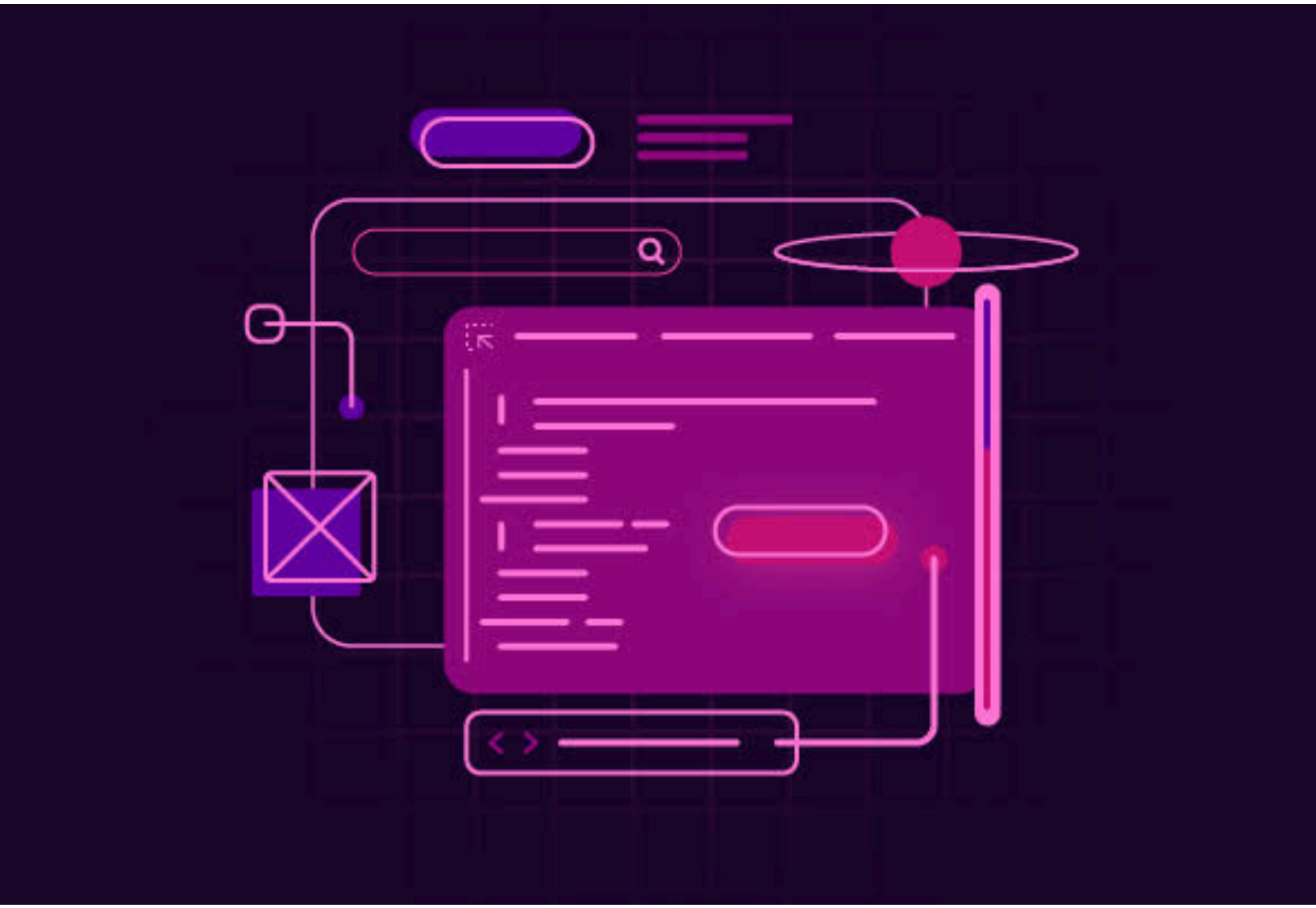
UNIVERSIDAD TECMILENIO

PROYECTO FINAL

Estructura de datos

Docente: Alberto Lemus Mera



Erick Rubio Lugo Galindo

UNIVERSIDAD TECMILENIO

ÍNDICE

Objetivo **1**
Instrucciones **1**
Código 2
Tarea.java 2
Empleado.java 2
AlgoritmoBusqueda.java 3
Arbolbinario.java 4
Grafo.java 5
HashNode.java 6
NodePrioridad.java 6
PriorityQueue.java 6
SimpleHashTable.java 8
App.java 9
Código corriendo **13**

UNIVERSIDAD TECMILENIO

Objetivo

Diseñar e implementar un sistema de gestión de tareas mediante estructuras de datos avanzadas y algoritmos de ordenamiento y búsqueda para optimizar las operaciones de asignación, consulta, y organización de tareas.

Instrucciones

1. **Preparación:**
    1. Revisa los conceptos de las estructuras de datos y algoritmos mencionados.
    2. Revisar la documentación de la fase I del proyecto.
2. **Implementación de la cola de prioridades:**
    1. Crea una cola de prioridades para gestionar las tareas con base en su urgencia y fecha de entrega.
3. **Uso de árboles binarios:**
    1. Implementa árboles binarios para organizar los empleados por departamentos y optimizar la búsqueda de asignaciones .
4. **Recursividad y algoritmos “divide y vencerás”:**
    1. Aplica la recursividad para calcular estadísticas de las tareas, como el tiempo estimado para completarlas.
    2. Utiliza el concepto de “divide y vencerás” para desarrollar un algoritmo que optimice la distribución de tareas en proyectos grandes .
5. **Tablas *hash* y clase *hashmap*:**
    1. Emplea tablas *hash* para almacenar y acceder rápidamente a la información de tareas y empleados.
6. **Métodos de ordenamiento y búsqueda:**
    1. Implementa métodos de ordenamiento para organizar las tareas por prioridad y fecha de entrega.
    2. Desarrolla métodos de búsqueda eficientes para encontrar tareas específicas.
7. **Grafos:**
    1. Utiliza grafos para representar las dependencias entre tareas en proyectos complejos.

Código

Tarea.java

```
public class Tarea {
   public String id;
   public String descripcion;
   public String departamento;
   public int urgencia;
   public int tiempoEstimado;

   public Tarea(String id, String descripcion, String departamento, int
urgencia, int tiempoEstimado) {
       this.id = id;
       this.descripcion = descripcion;
       this.departamento = departamento;
       this.urgencia = urgencia;
       this.tiempoEstimado = tiempoEstimado;
   }

   @Override
   public String toString() {
       return "[" + id + "] " + descripcion + " (Depto: " + departamento + ")
| Urgencia: " + urgencia + " | Tiempo: " + tiempoEstimado + " min";
   }
}
```

Empleado.java

```
public class Empleado {
   public int idMedico;
   public String nombre;
   public String departamento;

   public Empleado(int idMedico, String nombre, String departamento) {
       this.idMedico = idMedico;
       this.nombre = nombre;
       this.departamento = departamento;
   }

   @Override
   public String toString() {
       return "ID: " + idMedico + " | Nombre: " + nombre + " | Depto: " +
departamento;
   }
}
```

AlgoritmoBusqueda.java

```java
public class AlgortimosBusqueda {
   public static int buscarLineal(int[] arreglo, int objetivo) {
       for(int i = 0; i < arreglo.length; i++) {
           if (arreglo[i] == objetivo) {
               return i;
           }
       }
       return -1;
   }

   // Busqueda binaria recursiva O (Log n)
   public static int busquedaBinaria(int[] arreglo, int izquierda, int
derecha, int objetivo){
       if (izquierda > derecha) {
           return -1;
       }
       int medio = izquierda + (derecha - izquierda)/2;
       if (arreglo[medio] == objetivo) {
           return medio;
       }
       if (objetivo < arreglo[medio]) {
           return busquedaBinaria(arreglo, izquierda, medio-1, objetivo);
       }

       return busquedaBinaria(arreglo, medio+1, derecha, objetivo);
   }

   // NUEVO: Algoritmo Divide y Vencerás para calcular tiempos de tareas
   public static int calcularTiempoRecursivo(Tarea[] arreglo, int izquierda,
int derecha) {
       if (izquierda == derecha) {
           return arreglo[izquierda].tiempoEstimado;
       }
       int medio = izquierda + (derecha - izquierda) / 2;

       int tiempoIzq = calcularTiempoRecursivo(arreglo, izquierda, medio);
       int tiempoDer = calcularTiempoRecursivo(arreglo, medio + 1, derecha);

       return tiempoIzq + tiempoDer;
   }
}
```

Arbolbinario.java

```
public class Arbolbinario {
    public static class Nodo {
        Empleado data;
        Nodo left;
        Nodo right;

        public Nodo(Empleado data) {
            this.data = data;
            this.left = null;
            this.right = null;
        }
    }

    private Nodo raiz;

    public Arbolbinario() {
        this.raiz = null;
    }

    private Nodo insertarRecursivo(Nodo actual, Empleado dato) {
        if (actual == null) {
            return new Nodo(dato);
        }
        if (dato.idMedico < actual.data.idMedico) {
            actual.left = insertarRecursivo(actual.left, dato);
        } else if (dato.idMedico >= actual.data.idMedico) {
            actual.right = insertarRecursivo(actual.right, dato);
        }
        return actual;
    }

    public void insertar(Empleado dato) {
        raiz = insertarRecursivo(raiz, dato);
    }

    // --- NUEVOS MÉTODOS PARA LA INTERFAZ GRÁFICA ---
    public String obtenerInOrdenGUI() {
        StringBuilder resultado = new StringBuilder();
        inOrdenAcumulado(raiz, resultado);
        return resultado.toString();
    }

    private void inOrdenAcumulado(Nodo nodo, StringBuilder sb) {
        if (nodo == null) return;
        inOrdenAcumulado(nodo.left, sb);
        sb.append(nodo.data.toString()).append("\n"); // Acumulamos en lugar
de imprimir
        inOrdenAcumulado(nodo.right, sb);
    }
}
```

Grafo.java

```
import java.util.*;

public class Grafo {

    private Map<String, List<String>> listaAdyacencia = new HashMap<>();

    public void agregarVertice(String vertice) {
        listaAdyacencia.putIfAbsent(vertice, new ArrayList<>());
    }

    public void agregarArista(String origen, String destino) {
        agregarVertice(origen);
        agregarVertice(destino);
        listaAdyacencia.get(origen).add(destino);
    }

    // Consultar las dependencias directas de una tarea
    public String obtenerInfoDependencias(String idTarea) {
        List<String> prerequisitos = new ArrayList<>();

        // BQue tarea debe hacerse antes
        for (String vertice : listaAdyacencia.keySet()) {
            if (listaAdyacencia.get(vertice).contains(idTarea)) {
                prerequisitos.add(vertice);
            }
        }

        // Que tarea se desbloquea despues
        List<String> siguientes = listaAdyacencia.get(idTarea);

        String info = "";
        if (!prerequisitos.isEmpty()) {
            info += "  >> [REQUIERE PREVIO: " + String.join(", ",
prerequisitos) + "]";
        }
        if (siguientes != null && !siguientes.isEmpty()) {
            info += "  >> [HABILITA A: " + String.join(", ", siguientes) + "]";
        }

        return info;
    }

    // Obtener la lista que se hizo antes de prerequisitos
    public String[] obtenerPrerequisitosArreglo(String idTarea) {
        // Contar cuántos prerrequisitos hay para darle tamaño al arreglo
        int contador = 0;
        for (String vertice : listaAdyacencia.keySet()) {
            if (listaAdyacencia.get(vertice).contains(idTarea)) {
                contador++;
            }
        }

        // Crear el arreglo estático con el tamaño exacto
        String[] prerequisitos = new String[contador];
        int indice = 0;
```

```
        // Llenar el arreglo el arreglo
        for (String vertice : listaAdyacencia.keySet()) {
            if (listaAdyacencia.get(vertice).contains(idTarea)) {
                prerequisitos[indice] = vertice;
                indice++;
            }
        }
        return prerequisitos;
    }

    //
    public String obtenerRutaDfsGUI(String inicio) {
        Set<String> visitados = new HashSet<>();
        StringBuilder resultado = new StringBuilder();
        dfsAcumulado(inicio, visitados, resultado);
        return resultado.toString();
    }

    private void dfsAcumulado(String actual, Set<String> visitados,
StringBuilder sb) {
        visitados.add(actual);
        sb.append(" -> ").append(actual).append("\n");

        for(String vecino : listaAdyacencia.get(actual)) {
            if (!visitados.contains(vecino)) {
               dfsAcumulado(vecino, visitados, sb);
            }
        }
    }
}
```

HashNode.java

```
public class HashNode <K, V> {
    public K key;
    public V value;
    public HashNode<K, V> next;

    public HashNode(K key, V value) {
        this.key = key;
        this.value = value;
        this.next = null;
    };
}
```

NodePrioridad.java

```
public class NodePrioridad<T> {
 public T data;
 public NodePrioridad<T> next;
 public int prioridad;

 public NodePrioridad(T data, int prioridad) {
   this.data = data;
   this.prioridad = prioridad;
```

```
    this.next = null;
  }
}
```

PriorityQueue.java

```
public class PriorityQueue<T> {
    public NodePrioridad<T> front;
    public int size;

    public PriorityQueue() {
        this.front = null;
        this.size = 0;
    }

    public boolean isEmpty() {
        return front == null;
    }

    public void enqueue(T data, int prioridad) {
        NodePrioridad<T> nuevo = new NodePrioridad<>(data, prioridad);
        // Verificamos si la cola esta vacia o si el nueva elemento
        // tiene mayor prioridad
        if (isEmpty() || prioridad < front.prioridad) {
            nuevo.next = front;
            front = nuevo;
        } else {
            // Buscar el lugar correcto del elemento
            NodePrioridad<T> actual = front;
            while (actual.next != null && actual.next.prioridad <= prioridad)
{
                actual = actual.next;
            }
            // Insertar el nodo en el lugar correcto
            nuevo.next = actual.next;
            actual.next = nuevo;
        }
        size++;
        System.out.println("Elemento agregado correctamente");
    }

    // Metodo dequeue
    public T dequeue() {
        if (isEmpty()) {
            System.out.println("Nada que atender");
            return null;
        }
        T atendido = front.data;
        front = front.next;
        size--;
        System.out.println("Atendiendo el elemento con mayor priridad");
        return atendido;
    }

    public T peek() {
        if (isEmpty()) {
```

```
            System.out.println("No hay nada que mostrar");
            return null;
        }
        return front.data;
    }

    public void mostrar() {
        if (isEmpty()) {
            System.out.println("Nada que mostrar");
            return;
        }
        NodePrioridad<T> actual = front;
        System.out.println("Frente -> ");
        while (actual != null) {
            System.out.println(actual.data + " -> ");
            actual = actual.next;
        }
        System.out.println("FIN");
    }
}
```

SimpleHashTable.java

```
public class SimpleHashTable<K, V> {
    public HashNode<K, V> [] buckets;
    public int capacity;
    public int size;

    @SuppressWarnings("unchecked")
    public SimpleHashTable(int capacity) {
        this.capacity = capacity;
        this.buckets = (HashNode<K, V>[]) new HashNode[capacity];
        this.size = 0;
    }

    /*
    FUNCION DISPERCION(Hash function)
    Convierte la llave en un indice valido dentro del tamaño del arreglo
    */

    public int getBucketIndex(K key) {
        /*Funcion nativa de java para generar codigo con hash*/
        int hashCode = key.hashCode();
        /*Operacion modulo para ajustar al rango del arreglo*/
        return Math.abs(hashCode) % capacity;
    }

    /* INSERTAR
    Agrega o actualiza un valor asociado a una llave (push, enqueue)
    */

    public void put(K key, V value) {
        int BucketIndex = getBucketIndex(key);
        HashNode<K, V> head = buckets[BucketIndex];
        //Buscar la llave si ya existe para actualizar su valor
        HashNode<K, V> current = head;
```

```
        while (current != null) {
            if (current.key.equals(key)) {
                current.value = value;
                return;
            }
            current = current.next;
        }
        // Si la llave no existe insertamos un nuevo nodo
        size++;
        HashNode<K, V> newNode = new HashNode<>(key, value);
        newNode.next = head;
        buckets[BucketIndex] = newNode;
    }

    // Buscar un valor a traves de su llave
    public V getV(K key) {
        // Aqui guardamos la posicion de la llave [0,1,2,3,4]
        int bucketIndex = getBucketIndex(key);
        // Guardamos el nodo de la posicion de la llave
        HashNode<K, V> actual = buckets[bucketIndex];
        // Recorrer la lista con un while
        while (actual != null) {
            if (actual.key.equals(key)) {
                return actual.value;
            }
            actual = actual.next;
        }
        System.out.println("Elemento no encontrado");
        return null;
    }

    // Mostrar
    public void showTable() {
        for(int i = 0; i < capacity; i++){
            System.out.print("Registro " + i + ": ");
            HashNode<K, V> current = buckets[i];
            if (current == null) {
                System.out.println("Vacio");
            } else {
                System.out.print(" -> ");
                while (current != null) {
                    System.out.print("[ " + current.key + " : " + current.value
+ " ] -> ");
                    current = current.next;
                }
                System.out.println();
            }
        }
    }
}
```

App.java

```
import javax.swing.*;
import java.awt.*;
```

```
public class App extends JFrame {
   private JTextArea areaResultado;

   private PriorityQueue<Tarea> colaTareas;
   private SimpleHashTable<String, Tarea> directorioTareas;
   private Arbolbinario arbolMedicos;
   private Grafo grafoHospital;

   // Contador para los IDs
   private int contadorId = 6;

   // AArreglo para ver que tareas fueron terminadas
   private String[] tareasTerminadas;
   private int terminadasCount;

   public App() {
       colaTareas = new PriorityQueue<>();
       directorioTareas = new SimpleHashTable<>(20);
       arbolMedicos = new Arbolbinario();
       grafoHospital = new Grafo();

       tareasTerminadas = new String[500];
       terminadasCount = 0;

       // Empleados de prueba
       arbolMedicos.insertar(new Empleado(105, "Dr. Perez", "Urgencias"));
       arbolMedicos.insertar(new Empleado(101, "Dra. Gomez", "Cardiologia"));
       arbolMedicos.insertar(new Empleado(110, "Enf. Martinez", "Pediatria"));

       // Tareas de prueba
       Tarea t1 = new Tarea("T-001", "Paciente con paro cardíaco",
"Urgencias", 1, 60);
       colaTareas.enqueue(t1, t1.urgencia);
       directorioTareas.put(t1.id, t1);

       Tarea t2 = new Tarea("T-002", "Revisión de rutina", "Consultorio 3", 3,
15);
       colaTareas.enqueue(t2, t2.urgencia);
       directorioTareas.put(t2.id, t2);

       Tarea t3 = new Tarea("T-003", "Curación de herida leve", "Enfermería",
2, 20);
       colaTareas.enqueue(t3, t3.urgencia);
       directorioTareas.put(t3.id, t3);

       Tarea t4 = new Tarea("T-004", "Cirugía de apéndice", "Quirófano", 1,
120);
       colaTareas.enqueue(t4, t4.urgencia);
       directorioTareas.put(t4.id, t4);

       Tarea t5 = new Tarea("T-005", "Traslado a Rayos X", "Radiología", 3,
10);
       colaTareas.enqueue(t5, t5.urgencia);
       directorioTareas.put(t5.id, t5);

       // Dependecia de prueba
       grafoHospital.agregarArista("T-005", "T-004");
```

```
        grafoHospital.agregarArista("T-002", "T-003");

        // Interfaz
        setTitle("Sistema del Hospital General");
        setSize(700, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JPanel panelPrincipal = new JPanel(new BorderLayout(15, 15));
        panelPrincipal.setBorder(BorderFactory.createEmptyBorder(15, 15, 15,
15));

        JPanel panelIzquierdo = new JPanel(new BorderLayout());
        JPanel panelBotones = new JPanel(new GridLayout(7, 1, 5, 10));

        JButton btnAgregar = new JButton("Agregar Tarea");
        JButton btnAtender = new JButton("Atender Siguiente");
        JButton btnBuscar = new JButton("Buscar Tarea por ID");
        JButton btnMostrar = new JButton("Mostrar Todas");
        JButton btnVincular = new JButton("Crear Dependencia (Grafo)");
        JButton btnCalcular = new JButton("Calcular Tiempo Total");
        JButton btnPersonal = new JButton("Mostrar Personal");

        panelBotones.add(btnAgregar);
        panelBotones.add(btnAtender);
        panelBotones.add(btnBuscar);
        panelBotones.add(btnMostrar);
        panelBotones.add(btnVincular);
        panelBotones.add(btnCalcular);
        panelBotones.add(btnPersonal);

        panelIzquierdo.add(panelBotones, BorderLayout.NORTH);
        panelPrincipal.add(panelIzquierdo, BorderLayout.WEST);

        areaResultado = new JTextArea();
        areaResultado.setEditable(false);
        areaResultado.setFont(new Font("Arial", Font.PLAIN, 14));
        areaResultado.setText("Bienvenido al sistema del hospital.\n(Se han
cargado 5 tareas de prueba en el sistema).\n\nSeleccione una opción del menú
izquierdo.");
        panelPrincipal.add(new JScrollPane(areaResultado),
BorderLayout.CENTER);

        add(panelPrincipal);

        // Botones

        // Agregar Tarea
        btnAgregar.addActionListener(e -> {
            JTextField txtDesc = new JTextField();
            JTextField txtDepto = new JTextField();
            JTextField txtUrgencia = new JTextField("1");
            JTextField txtTiempo = new JTextField("30");

            Object[] formulario = {
                "Descripción de la tarea:", txtDesc,
                "Departamento:", txtDepto,
```

```
                "Urgencia (1=Alta, 2=Media, 3=Baja):", txtUrgencia,
                "Tiempo Estimado (minutos):", txtTiempo
            };

            int opcion = JOptionPane.showConfirmDialog(App.this, formulario,
"Registrar Nueva Tarea", JOptionPane.OK_CANCEL_OPTION);

            if (opcion == JOptionPane.OK_OPTION) {
                try {
                    String desc = txtDesc.getText().trim();
                    String depto = txtDepto.getText().trim();
                    int urg = Integer.parseInt(txtUrgencia.getText().trim());
                    int tiempo = Integer.parseInt(txtTiempo.getText().trim());

                    if(desc.isEmpty() || depto.isEmpty()) {
                        areaResultado.setText("Error: Por favor llena todos los
campos de texto.");
                        return;
                    }

                    String idAutomatico = String.format("T-%03d", contadorId);
                    contadorId++;

                    Tarea nueva = new Tarea(idAutomatico, desc, depto, urg,
tiempo);
                    colaTareas.enqueue(nueva, urg);
                    directorioTareas.put(idAutomatico, nueva);

                    areaResultado.setText("Tarea agregada exitosamente:\n" +
nueva.toString());
                } catch (NumberFormatException ex) {
                    areaResultado.setText("Error: Urgencia y Tiempo deben ser
NÚMEROS.");
                }
            }
        });

        // Atender Tarea
        btnAtender.addActionListener(e -> {
            if (colaTareas.isEmpty()) {
                areaResultado.setText("No hay tareas pendientes por atender en
la cola.");
                return;
            }

            Tarea[] tareasBloqueadas = new Tarea[colaTareas.size];
            int bloqueadasCount = 0;
            Tarea tareaAAtender = null;

            while (!colaTareas.isEmpty()) {
                Tarea actual = colaTareas.dequeue();

                String[] prerequisitos =
grafoHospital.obtenerPrerequisitosArreglo(actual.id);
                boolean estaLista = true;

                for (int i = 0; i < prerequisitos.length; i++) {
```

```
                    String pre = prerequisitos[i];
                    boolean preTerminado = false;

                    for (int j = 0; j < terminadasCount; j++) {
                        if (tareasTerminadas[j].equals(pre)) {
                            preTerminado = true;
                            break;
                        }
                    }

                    if (!preTerminado) {
                        estaLista = false;
                        break;
                    }
                }

                if (estaLista) {
                    tareaAAtender = actual;
                    break;
                } else {
                    tareasBloqueadas[bloqueadasCount] = actual;
                    bloqueadasCount++;
                }
            }

            for (int i = 0; i < bloqueadasCount; i++) {
                colaTareas.enqueue(tareasBloqueadas[i],
tareasBloqueadas[i].urgencia);
            }

            if (tareaAAtender != null) {
                tareasTerminadas[terminadasCount] = tareaAAtender.id;
                terminadasCount++;
                areaResultado.setText("Atendiendo la tarea (Prerrequisitos
verificados):\n\n" + tareaAAtender.toString());
            } else {
                areaResultado.setText("¡ATENCIÓN!\nNo se puede atender la tarea
más urgente en este momento.\n\n" +
                                      "Está bloqueada porque depende de un
procedimiento que aún no se ha realizado.");
            }
        });

        // 3. Buscar por ID (Tabla Hash)
        btnBuscar.addActionListener(e -> {
            String idBusqueda = JOptionPane.showInputDialog(App.this, "Ingrese
el ID de la tarea a buscar (Ej. T-001):");
            if(idBusqueda != null && !idBusqueda.trim().isEmpty()) {
                Tarea encontrada = directorioTareas.getV(idBusqueda.trim());
                if (encontrada != null) {
                    areaResultado.setText("Resultado de la Búsqueda:\n" +
encontrada.toString());
                } else {
                    areaResultado.setText("La tarea con ID '" + idBusqueda + "'
no existe en el sistema.");
                }
            }
```

```
        });

        // Mostrar Tareas
        btnMostrar.addActionListener(e -> {
            if (colaTareas.isEmpty()) {
                areaResultado.setText("No hay tareas pendientes en el
sistema.");
            } else {
                StringBuilder texto = new StringBuilder("--- TODAS LAS TAREAS
PENDIENTES ---\n");
                texto.append("(Ordenadas por mayor urgencia)\n\n");

                NodePrioridad<Tarea> actual = colaTareas.front;
                while(actual != null) {
                    texto.append(actual.data.toString());
                    String dependencias =
grafoHospital.obtenerInfoDependencias(actual.data.id);
                    texto.append(dependencias).append("\n\n");
                    actual = actual.next;
                }
                areaResultado.setText(texto.toString());
            }
        });

        // Vincular Tareas
        btnVincular.addActionListener(e -> {
            JTextField txtPre = new JTextField();
            JTextField txtDep = new JTextField();

            Object[] formulario = {
                "ID de la Tarea Prerrequisito (Ej. T-005):", txtPre,
                "ID de la Tarea que depende de la anterior (Ej. T-004):",
txtDep
            };

            int opcion = JOptionPane.showConfirmDialog(App.this, formulario,
"Crear Dependencia de Tareas", JOptionPane.OK_CANCEL_OPTION);

            if (opcion == JOptionPane.OK_OPTION) {
                String idPre = txtPre.getText().trim();
                String idDep = txtDep.getText().trim();

                Tarea tPre = directorioTareas.getV(idPre);
                Tarea tDep = directorioTareas.getV(idDep);

                if (tPre == null || tDep == null) {
                    areaResultado.setText("Error: Uno o ambos IDs no existen en
el sistema.");
                } else {
                    grafoHospital.agregarArista(idPre, idDep);
                    areaResultado.setText("Dependencia creada con éxito en el
Grafo:\n\n" +
                                          "Para poder iniciar [" + idDep + "] "
+ tDep.descripcion + "...\n" +
                                          "Primero debe completarse [" + idPre
+ "] " + tPre.descripcion);
                }
```

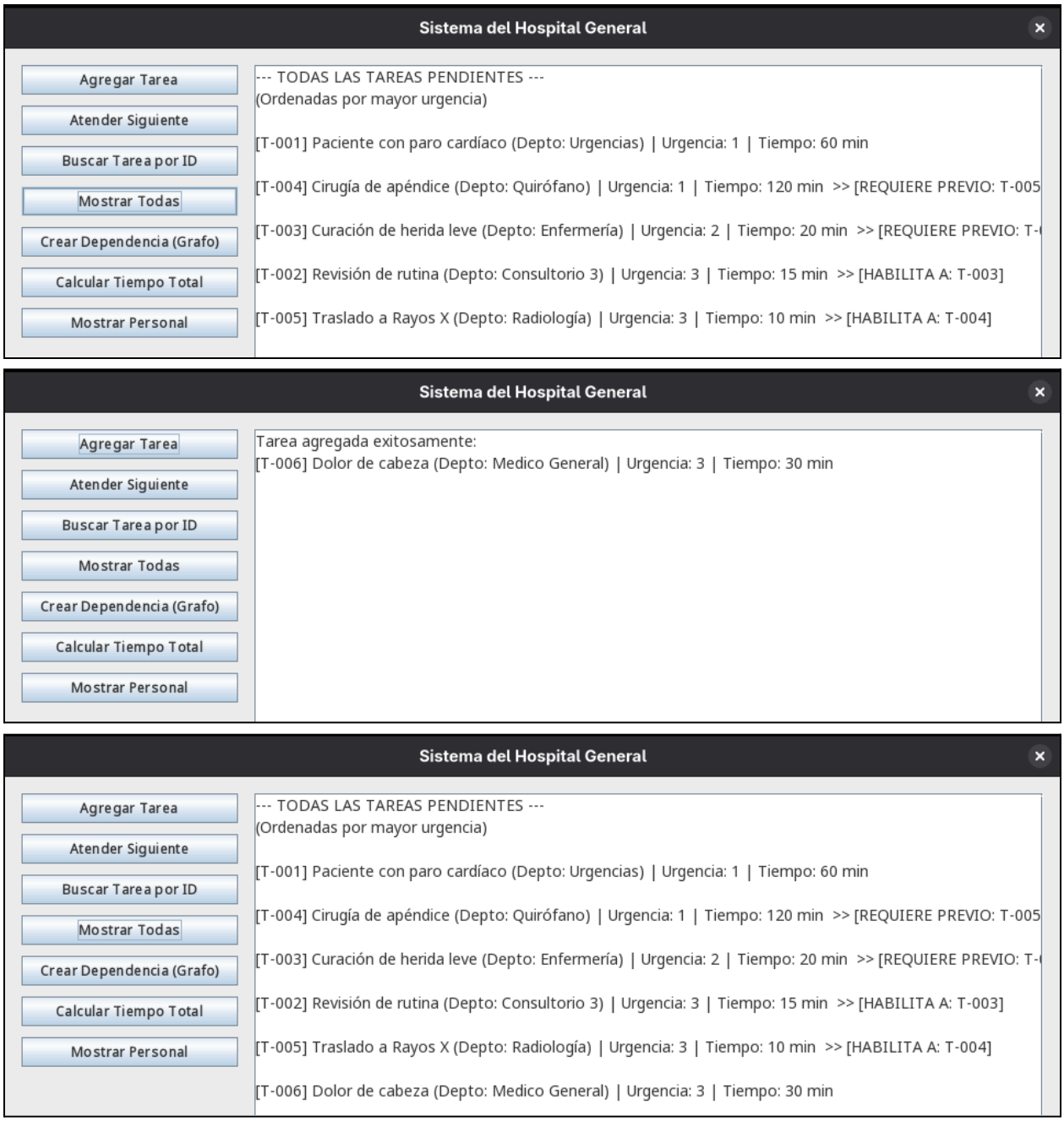
```
            }
        });

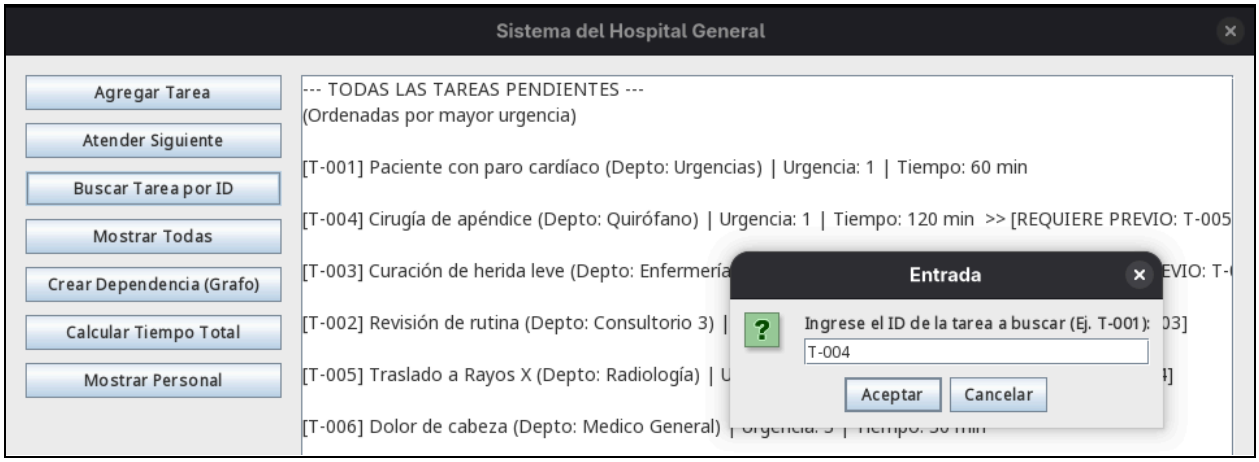
        // Calcular Tiempo Total
        btnCalcular.addActionListener(e -> {
            if (colaTareas.isEmpty()) {
                areaResultado.setText("La cola está vacía. No hay tiempos que
calcular.");
            } else {
                Tarea[] arreglo = new Tarea[colaTareas.size];
                NodePrioridad<Tarea> actual = colaTareas.front;
                int i = 0;
                while(actual != null) {
                    arreglo[i] = actual.data;
                    actual = actual.next;
                    i++;
                }
                int total = AlgortimosBusqueda.calcularTiempoRecursivo(arreglo,
0, arreglo.length - 1);
                areaResultado.setText("Cálculo realizado (Divide y
Vencerás).\nTiempo total estimado para todas las tareas: " + total + "
minutos.");
            }
        });

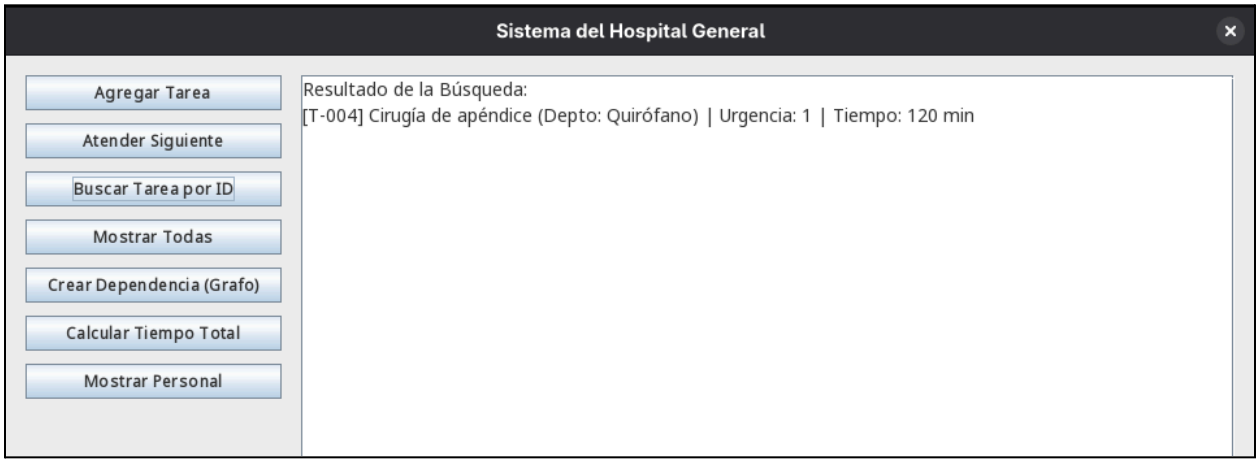
        // Mostrar Personal
        btnPersonal.addActionListener(e -> {
            String resultado = arbolMedicos.obtenerInOrdenGUI();
            if (resultado.isEmpty()) {
                areaResultado.setText("No hay personal registrado en el
árbol.");
            } else {
                areaResultado.setText("--- PERSONAL MÉDICO (Ordenado por ID)
---\n\n" + resultado);
            }
        });
    }

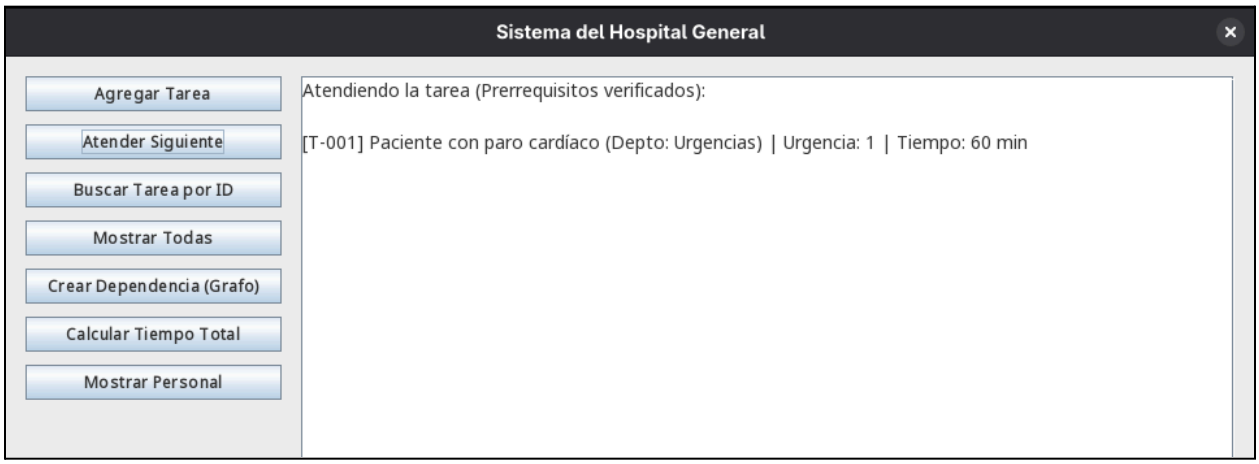
    public static void main(String[] args) {
        App ventana = new App();
        ventana.setVisible(true);
    }
}
```

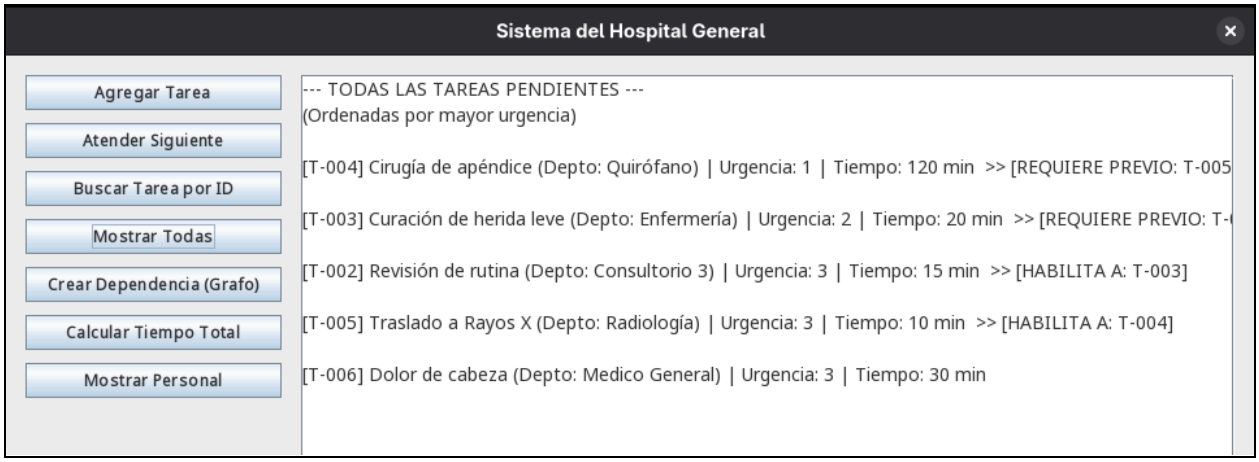
Código corriendo

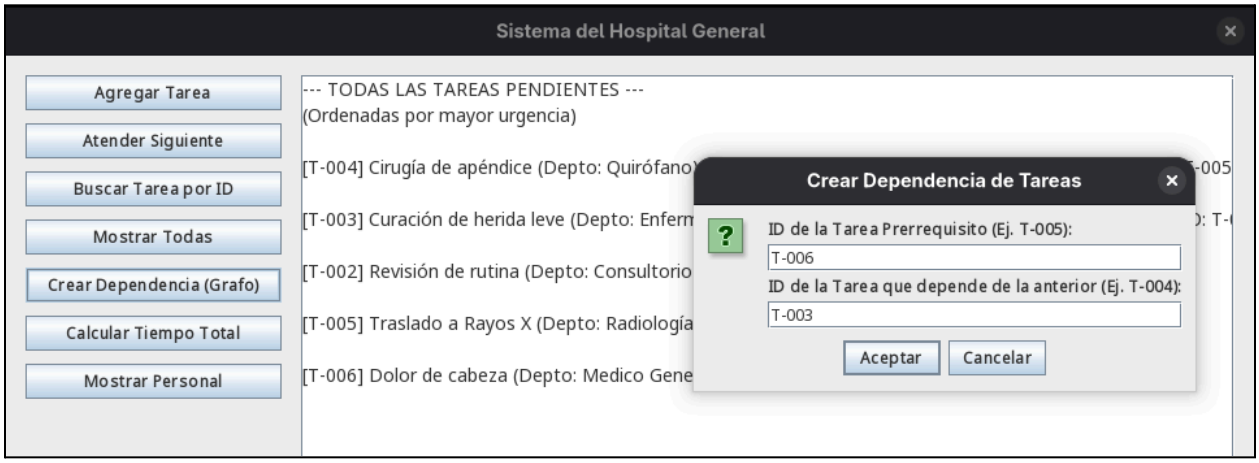

Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
--- TODAS LAS TAREAS PENDIENTES ---
(Ordenadas por mayor urgencia)
[T-001] Paciente con paro cardíaco (Depto: Urgencias) | Urgencia: 1 | Tiempo: 60 min
[T-004] Cirugía de apéndice (Depto: Quirófano) | Urgencia: 1 | Tiempo: 120 min >> [REQUIERE PREVIO: T-005
[T-003] Curación de herida leve (Depto: Enfermería) | Urgencia: 2 | Tiempo: 20 min >> [REQUIERE PREVIO: T-
[T-002] Revisión de rutina (Depto: Consultorio 3) | Urgencia: 3 | Tiempo: 15 min >> [HABILITA A: T-003]
[T-005] Traslado a Rayos X (Depto: Radiología) | Urgencia: 3 | Tiempo: 10 min >> [HABILITA A: T-004]
Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
Tarea agregada exitosamente:
[T-006] Dolor de cabeza (Depto: Medico General) | Urgencia: 3 | Tiempo: 30 min
Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
--- TODAS LAS TAREAS PENDIENTES ---
(Ordenadas por mayor urgencia)
[T-001] Paciente con paro cardíaco (Depto: Urgencias) | Urgencia: 1 | Tiempo: 60 min
[T-004] Cirugía de apéndice (Depto: Quirófano) | Urgencia: 1 | Tiempo: 120 min >> [REQUIERE PREVIO: T-005
[T-003] Curación de herida leve (Depto: Enfermería) | Urgencia: 2 | Tiempo: 20 min >> [REQUIERE PREVIO: T-
[T-002] Revisión de rutina (Depto: Consultorio 3) | Urgencia: 3 | Tiempo: 15 min >> [HABILITA A: T-003]
[T-005] Traslado a Rayos X (Depto: Radiología) | Urgencia: 3 | Tiempo: 10 min >> [HABILITA A: T-004]
[T-006] Dolor de cabeza (Depto: Medico General) | Urgencia: 3 | Tiempo: 30 min

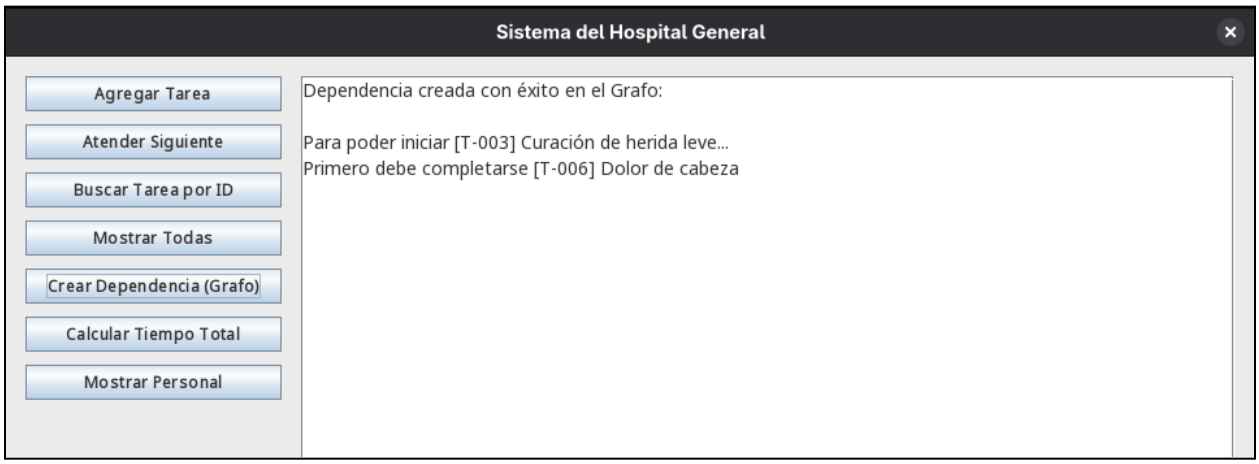

Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
--- TODAS LAS TAREAS PENDIENTES ---
(Ordenadas por mayor urgencia)
[T-001] Paciente con paro cardíaco (Depto: Urgencias) | Urgencia: 1 | Tiempo: 60 min
[T-004] Cirugía de apéndice (Depto: Quirófano) | Urgencia: 1 | Tiempo: 120 min >> [REQUIERE PREVIO: T-005
[T-003] Curación de herida leve (Depto: Enfermería
[T-002] Revisión de rutina (Depto: Consultorio 3) |
[T-005] Traslado a Rayos X (Depto: Radiología) |
[T-006] Dolor de cabeza (Depto: Medico General)
Entrada
Ingrese el ID de la tarea a buscar (Ej. T-001):
T-004
Aceptar
Cancelar

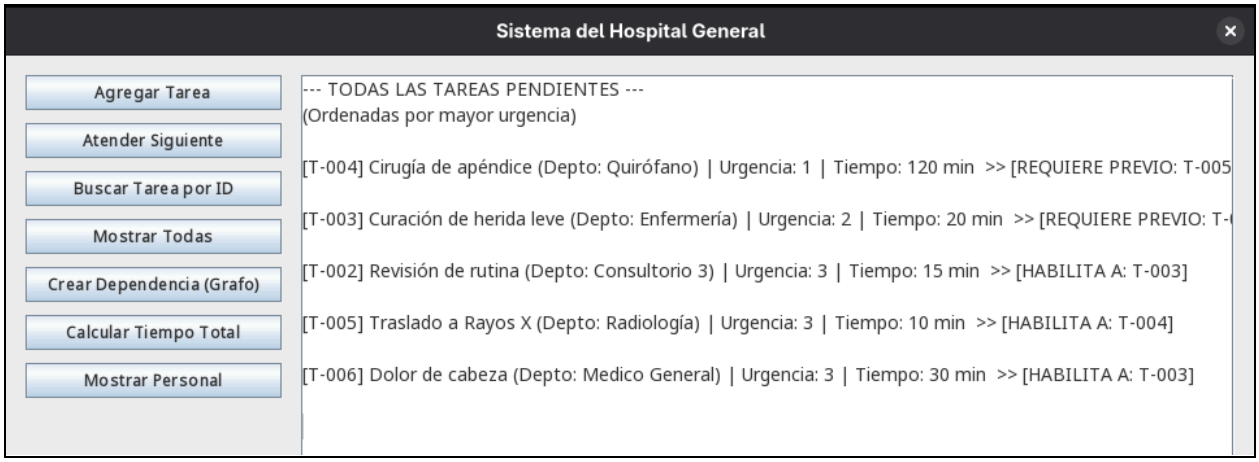

Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
Resultado de la Búsqueda:
[T-004] Cirugía de apéndice (Depto: Quirófano) | Urgencia: 1 | Tiempo: 120 min

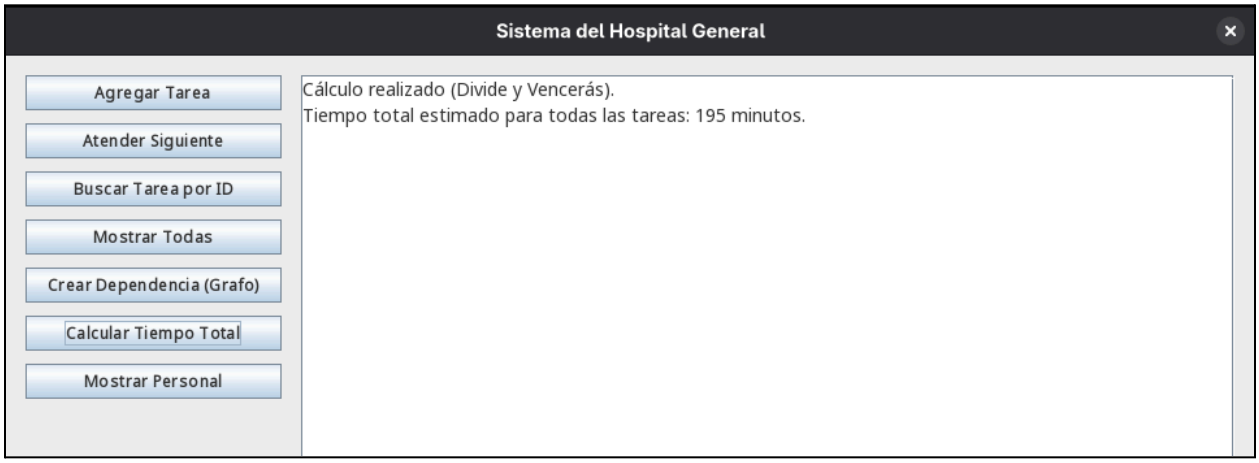

Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
Atendiendo la tarea (Prerrequisitos verificados):
[T-001] Paciente con paro cardíaco (Depto: Urgencias) | Urgencia: 1 | Tiempo: 60 min

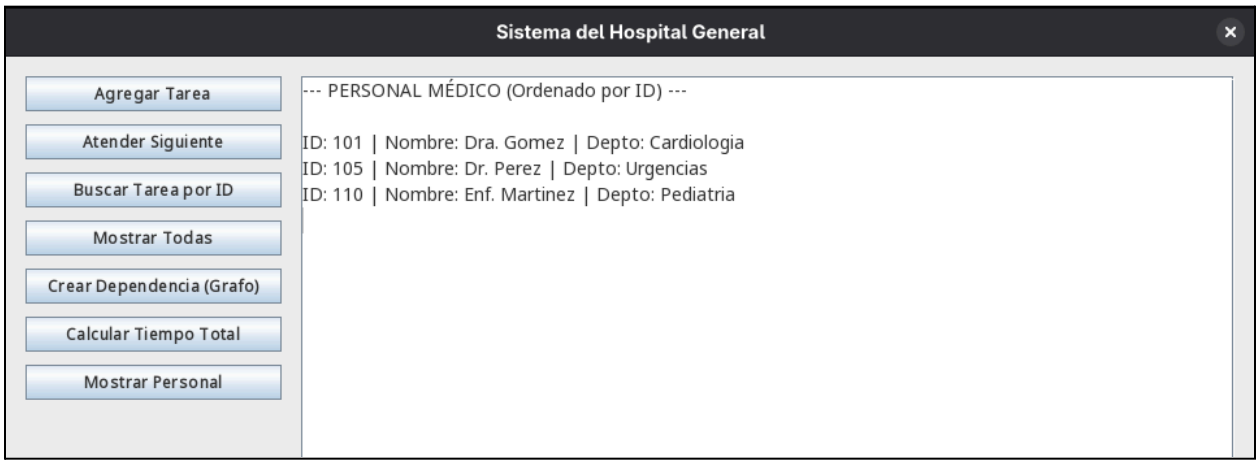

Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
--- TODAS LAS TAREAS PENDIENTES ---
(Ordenadas por mayor urgencia)
[T-004] Cirugía de apéndice (Depto: Quirófano) | Urgencia: 1 | Tiempo: 120 min >> [REQUIERE PREVIO: T-005
[T-003] Curación de herida leve (Depto: Enfermería) | Urgencia: 2 | Tiempo: 20 min >> [REQUIERE PREVIO: T-
[T-002] Revisión de rutina (Depto: Consultorio 3) | Urgencia: 3 | Tiempo: 15 min >> [HABILITA A: T-003]
[T-005] Traslado a Rayos X (Depto: Radiología) | Urgencia: 3 | Tiempo: 10 min >> [HABILITA A: T-004]
[T-006] Dolor de cabeza (Depto: Medico General) | Urgencia: 3 | Tiempo: 30 min


Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
--- TODAS LAS TAREAS PENDIENTES ---
(Ordenadas por mayor urgencia)
Crear Dependencia de Tareas
ID de la Tarea Prerrequisito (Ej. T-005):
T-006
ID de la Tarea que depende de la anterior (Ej. T-004):
T-003
Aceptar
Cancelar


Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
Dependencia creada con éxito en el Grafo:
Para poder iniciar [T-003] Curación de herida leve...
Primero debe completarse [T-006] Dolor de cabeza


Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
--- TODAS LAS TAREAS PENDIENTES ---
(Ordenadas por mayor urgencia)
[T-004] Cirugía de apéndice (Depto: Quirófano) | Urgencia: 1 | Tiempo: 120 min >> [REQUIERE PREVIO: T-005
[T-003] Curación de herida leve (Depto: Enfermería) | Urgencia: 2 | Tiempo: 20 min >> [REQUIERE PREVIO: T-
[T-002] Revisión de rutina (Depto: Consultorio 3) | Urgencia: 3 | Tiempo: 15 min >> [HABILITA A: T-003]
[T-005] Traslado a Rayos X (Depto: Radiología) | Urgencia: 3 | Tiempo: 10 min >> [HABILITA A: T-004]
[T-006] Dolor de cabeza (Depto: Medico General) | Urgencia: 3 | Tiempo: 30 min >> [HABILITA A: T-003]


Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
Cálculo realizado (Divide y Vencerás).
Tiempo total estimado para todas las tareas: 195 minutos.


Sistema del Hospital General
Agregar Tarea
Atender Siguiente
Buscar Tarea por ID
Mostrar Todas
Crear Dependencia (Grafo)
Calcular Tiempo Total
Mostrar Personal
--- PERSONAL MÉDICO (Ordenado por ID) ---
ID: 101 | Nombre: Dra. Gomez | Depto: Cardiologia
ID: 105 | Nombre: Dr. Perez | Depto: Urgencias
ID: 110 | Nombre: Enf. Martinez | Depto: Pediatria